

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH
KHOA CHẤT LƯỢNG CAO



BÁO CÁO BÀI TẬP LỚN

NHÓM K

ĐỀ TÀI: QLogic IQ - ỨNG DỤNG QUIZ HỌC TẬP ÔN LUYỆN KIẾN THỨC

Ngành: CÔNG NGHỆ THÔNG TIN

GVHD: Trương Quang Tuấn

Môn: Lập trình thiết bị di động

Sinh viên thực hiện: 3 thành viên

Lớp: 010412103404

Họ và tên	MSSV
Lê Văn Tùng	045205000787
Trần Phát Tài	080205011244
Phạm Xuân Dũng	046205000170

Mục lục

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI	5
1.1.Tên đề tài	5
1.2.Giới thiệu chung về ứng dụng	5
1.3.Lĩnh vực áp dụng	6
1.4.Đối tượng sử dụng	6
CHƯƠNG 2. GIỚI THIỆU SƠ LƯỢC VỀ CÔNG NGHỆ SỬ DỤNG	7
2.1.Ngôn ngữ và Framework	7
2.2.Nền tảng phát triển	7
2.3.Dịch vụ Backend – Firebase	7
2.4.Thư viện và công nghệ hỗ trợ	8
2.5.Cấu trúc dự án theo kiến trúc MVVM	8
CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG	9
3.1.Yêu cầu chức năng	9
3.2.Yêu cầu phi chức năng	10
3.3.Sơ đồ Use Case (Tổng quát)	10
3.4.Biểu đồ luồng dữ liệu (DFD - Level 0)	15
CHƯƠNG 4. THIẾT KẾ HỆ THỐNG	16
4.1.Thiết kế UI/UX	16
4.2.Thiết kế cơ sở dữ liệu (Firebase Realtime Database / Firestore)	25
4.3.Thiết kế luồng điều hướng (Navigation Flow)	26
4.4.Sơ đồ kiến trúc hệ thống	27
4.5.Sơ đồ lớp (Class Diagram)	29
4.6.Sơ đồ tuần tự (Sequence Diagram)	29

CHƯƠNG 5. TRIỂN KHAI VÀ CÀI ĐẶT	31
5.1.Môi trường phát triển	31
5.2.Thư viện sử dụng	31
5.3.Các bước triển khai ứng dụng	32
CHƯƠNG 6. KẾT QUẢ THỰC NGHIỆM	34
6.1.Hình ảnh giao diện thực tế	34
6.2.Các chức năng đã thực hiện được	35
6.3.Demo sản phẩm	35
CHƯƠNG 7. ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN	37
7.1.Uưu điểm của ứng dụng	37
7.2.Hạn chế	37
7.3.Hướng phát triển trong tương lai	37
CHƯƠNG 8. KẾT LUẬN	38
CHƯƠNG 9. TÀI LIỆU THAM KHẢO	39

LỜI NÓI ĐẦU

Thông tin nhóm

Nhóm thực hiện gồm 3 thành viên.

Git của nhóm: <https://github.com/tunght2005/app-mobile-QuizLogicIQ>

Mỗi người đảm nhiệm một vai trò cụ thể trong quá trình phát triển ứng dụng:

STT	Họ và tên	Vai trò	Ghi chú
1	Lê Văn Tùng	Trưởng nhóm – Quản lý, thiết kế UI/UX chính	Phụ trách phần giao diện
2	Phạm Xuân Dũng	Lập trình chức năng, xử lý API chính	Phụ trách API
3	Trần Phát Tài	Kiểm thử, xử lý logic và kết nối Firebase	Phụ trách dữ liệu

Lý do chọn đề tài

Trong thời đại hiện nay, việc rèn luyện tư duy logic và khả năng giải quyết vấn đề không chỉ giới hạn trong lĩnh vực học thuật mà còn đóng vai trò then chốt trong cuộc sống hàng ngày cũng như môi trường làm việc. Tuy nhiên, phần lớn người dùng, đặc biệt là học sinh, sinh viên, thiếu công cụ trực quan và thú vị để luyện tập kỹ năng này một cách đều đặn và hiệu quả.

Với mong muốn tạo ra một công cụ học tập sáng tạo, dễ tiếp cận và có khả năng nâng cao tư duy logic cho mọi đối tượng, nhóm chúng em quyết định thực hiện đề tài: **“Xây dựng ứng dụng rèn luyện trí tuệ – QLOGIC IQ”** trên nền tảng di động. Ứng dụng có giao diện thân thiện, dễ sử dụng cùng hệ thống bài tập được phân chia hợp lý, giúp người dùng luyện tập hàng ngày một cách chủ động, vui vẻ và hiệu quả hơn.

Mục tiêu

- Xây dựng ứng dụng di động hỗ trợ người dùng luyện tập các môn học cần thiết, IQ logic..
- Thiết kế giao diện thân thiện, phù hợp với đa dạng người dùng.
- Phát triển các tính năng như: đăng nhập, phân loại bài học (bài flashcard và bài test), lưu kết quả, thư viện đề,...
- Tích hợp hệ thống điểm số, theo dõi tiến độ.

Ý nghĩa thực tiễn

- Giúp người dùng cải thiện tư duy logic, kỹ năng học tập và phản xạ làm bài.
- Hỗ trợ học sinh, sinh viên ôn luyện các môn học một cách chủ động và hiệu quả.
- Góp phần thúc đẩy ứng dụng công nghệ vào giáo dục hiện đại.
- Tạo môi trường thực tiễn để sinh viên lập trình rèn luyện kỹ năng phát triển ứng dụng di động.

Phạm vi nghiên cứu

- Nền tảng: Thiết bị di động sử dụng hệ điều hành Android 7+.
- Đối tượng người dùng: học sinh, sinh viên và người yêu thích tư duy logic.
- Các chức năng nghiên cứu:
 - Đăng nhập, luyện tập, tạo lịch học, quản lý hồ sơ.
 - Giao diện người dùng (UI) theo kiến trúc MVVM.
 - Lưu trữ dữ liệu bằng Firebase.

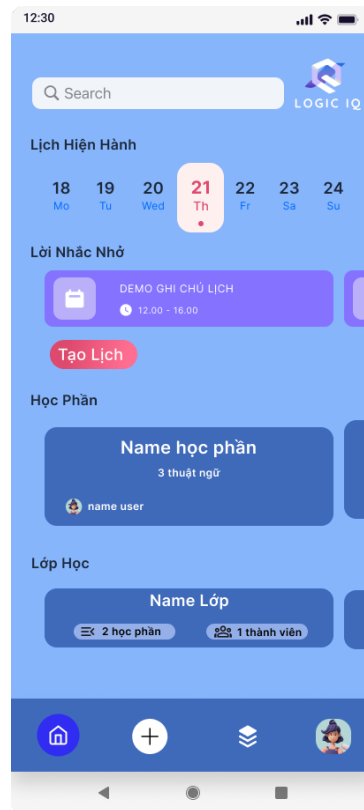
Phương pháp nghiên cứu

- Phân tích và thiết kế hệ thống sử dụng các mô hình: UML, DFD, MVVM.
- Thiết kế giao diện bằng Figma, lập trình bằng Jetpack Compose (Kotlin).
- Cài đặt và kiểm thử trên thiết bị ảo hoặc thật (Android).
- Tìm hiểu và ứng dụng các công nghệ: Firebase, Jetpack Compose, Kotlin.

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1. Tên đề tài

Xây dựng ứng dụng luyện tập tư duy logic – Logic IQ



Hình 1: Giao diện chính của ứng dụng Logic IQ

1.2. Giới thiệu chung về ứng dụng

QLogic IQ là một ứng dụng di động được phát triển với mục tiêu giúp người dùng rèn luyện tư duy logic, cải thiện phản xạ nhanh và khả năng suy luận thông qua hệ thống câu hỏi trắc nghiệm và các trò chơi trí tuệ đa dạng.

Ứng dụng được thiết kế với giao diện thân thiện, tối ưu cho trải nghiệm người dùng, cho phép người dùng tiếp cận dễ dàng với các bài tập theo từng cấp độ từ cơ bản đến nâng cao. Thông qua việc sử dụng Logic IQ thường xuyên, người dùng có thể cải thiện kỹ năng suy luận, tăng khả năng ghi nhớ và phản xạ trong các tình huống thực tiễn.

Các tính năng chính bao gồm:

- Đăng ký/đăng nhập tài khoản người dùng.
- Hệ thống bài tập đa dạng.

- Giao diện luyện tập đơn giản, dễ thao tác.
- Theo dõi tiến trình học tập và thống kê kết quả.
- Thử thách logic mỗi ngày để duy trì sự tập trung và tiến bộ liên tục.
- Hệ thống thông báo nhắc nhở theo lịch tạo.

1.3. Lĩnh vực áp dụng

Ứng dụng QLogic IQ thuộc lĩnh vực Công nghệ giáo dục (Educational Technology), cụ thể tập trung vào các mảng sau:

- Học tập kỹ năng mềm: phát triển tư duy, khả năng phân tích và ghi nhớ.
- Giáo dục hỗ trợ: hỗ trợ học sinh, sinh viên luyện tập, kiểm tra tư duy theo đề được thêm.
- Phát triển cá nhân: nâng cao tư duy, rèn luyện trí nhớ mỗi ngày.
- Giải trí có tính giáo dục: mang lại trải nghiệm vui vẻ nhưng vẫn giúp phát triển tư duy trí tuệ.

1.4. Đối tượng sử dụng

Ứng dụng hướng đến các nhóm người dùng cụ thể như sau:

- Học sinh, sinh viên: hỗ trợ trong việc phát triển tư duy logic, chuẩn bị cho các kỳ thi, kiểm tra hoặc luyện IQ.
- Người đi làm: tăng cường kỹ năng tư duy phản biện, phân tích và ra quyết định trong công việc.
- Người yêu thích trò chơi trí tuệ: trải nghiệm các dạng câu đố logic hấp dẫn, nâng cao khả năng tập trung và giải quyết vấn đề.
- Người đang luyện thi năng lực tư duy: luyện tập các bài test tư duy nhanh, luyện trắc nghiệm IQ, EQ,...

CHƯƠNG 2. GIỚI THIỆU SƠ LƯỢC VỀ CÔNG NGHỆ SỬ DỤNG

2.1. Ngôn ngữ và Framework

Kotlin là ngôn ngữ lập trình hiện đại được Google công nhận là ngôn ngữ chính thức cho phát triển Android. Kotlin hỗ trợ đầy đủ lập trình hướng đối tượng (OOP) và lập trình hàm (functional programming), với cú pháp ngắn gọn, an toàn null (null safety) và tương thích với mã Java. Việc sử dụng Kotlin giúp tăng tốc độ phát triển ứng dụng, giảm lỗi và dễ bảo trì.

Jetpack Compose là môi trường phát triển tích hợp (IDE) chính thức do Google cung cấp cho lập trình Android. IDE này hỗ trợ đầy đủ công cụ như code editor, layout editor, emulator, debug, profiler và Gradle build system. Android Studio cũng hỗ trợ Jetpack Compose, Hilt, Firebase và các thư viện khác thông qua các plugin và cấu hình để sử dụng.

2.2. Nền tảng phát triển

Android Studio

Android Studio là môi trường phát triển tích hợp (IDE) chính thức do Google cung cấp cho lập trình Android. IDE này hỗ trợ đầy đủ công cụ như trình chỉnh sửa mã, thiết kế giao diện, giả lập thiết bị, công cụ debug, profiler và hệ thống build Gradle. Android Studio cũng hỗ trợ Jetpack Compose, Hilt, Firebase và các thư viện khác thông qua các plugin và cấu hình để sử dụng.

2.3. Dịch vụ Backend – Firebase

Firebase là nền tảng Backend-as-a-Service (BaaS) do Google phát triển, giúp các lập trình viên Android xây dựng ứng dụng một cách nhanh chóng mà không cần triển khai máy chủ riêng. Trong dự án này, Firebase cung cấp các chức năng sau:

- **Firebase Authentication:** Hỗ trợ xác thực người dùng bằng nhiều phương thức như Email/Password, Google Sign-In, Số điện thoại (OTP), v.v. Giúp bảo mật quá trình đăng nhập và đăng ký.
- **Cloud Firestore:** Là cơ sở dữ liệu NoSQL theo thời gian thực, dùng để lưu trữ thông tin người dùng, bài đăng, tin nhắn,... Firestore hỗ trợ đồng bộ dữ liệu hai chiều (client server) và tự động cập nhật UI khi dữ liệu thay đổi.
- **Firebase Storage:** Lưu trữ các tệp dạng lớn như hình ảnh, video, file đính kèm. Firebase Storage tích hợp với Firebase Authentication và Firestore giúp kiểm soát quyền truy cập tệp dễ dàng.

- **Firebase Cloud Messaging (FCM):** (nếu có sử dụng) Cung cấp khả năng gửi thông báo đẩy đến người dùng theo lịch đã đặt hoặc sự kiện cụ thể. Ví dụ: thông báo đến giờ học, nhắc lịch làm bài,...

2.4. Thư viện và công nghệ hỗ trợ

- **Navigation Compose:** Thư viện điều hướng chính thức cho Jetpack Compose, giúp dễ dàng chuyển giữa các màn hình (screen) với cấu trúc rõ ràng và hỗ trợ truyền tham số, deep link.
- **Hilt:** Là thư viện Dependency Injection giúp quản lý vòng đời (lifecycle) của ViewModel, Repository và các lớp liên quan. Hilt giúp giảm bớt việc khởi tạo thủ công, tăng tính tái sử dụng và kiểm thử.
- **ViewModel + LiveData/StateFlow:** ViewModel lưu trữ và quản lý dữ liệu liên quan đến UI theo vòng đời. LiveData và StateFlow giúp UI tự động cập nhật khi dữ liệu thay đổi, phù hợp với kiến trúc MVVM.
- **Coil:** Là thư viện dùng để tải ảnh từ internet hoặc từ Firebase Storage và hiển thị lên Compose. Coil được thiết kế cho Kotlin Coroutines nên tích hợp rất tốt với Compose.
- **Coroutines:** Kotlin Coroutines giúp xử lý bất đồng bộ dễ dàng, tránh callback hell. Được sử dụng để gọi API, xử lý Firebase hoặc thao tác dữ liệu mà không chặn giao diện.

2.5. Cấu trúc dự án theo kiến trúc MVVM

- **Model:** Là các lớp dữ liệu đại diện cho thông tin thực thể trong ứng dụng như User, Quiz, Question, Answer, Reminder, Class,... Model không chứa logic giao diện mà chỉ là dữ liệu thuần.
- **View (UI):** Là các màn hình và thành phần giao diện được viết bằng Jetpack Compose. View hiển thị dữ liệu từ ViewModel và phản hồi tương tác người dùng (click, nhập liệu...).
- **ViewModel:** Là lớp trung gian giữa UI và dữ liệu. ViewModel xử lý logic nghiệp vụ, chuyển đổi dữ liệu trước khi đưa lên UI. ViewModel có thể sử dụng LiveData hoặc StateFlow để thông báo cho View khi có sự thay đổi.
- **Repository:** Là tầng giao tiếp với Firebase hoặc các nguồn dữ liệu khác (REST API, Room,...). Repository giúp tách biệt logic dữ liệu khỏi ViewModel, dễ kiểm thử và tái sử dụng.

CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG

3.1. Yêu cầu chức năng

Hệ thống cần đảm bảo các chức năng chính sau:

STT	Chức năng	Mô tả
1	Đăng ký / Đăng nhập	Cho phép người dùng tạo tài khoản, đăng nhập bằng email hoặc Google.
2	Tạo bài kiểm tra (bài thi)	Người dùng luyện tập các bài theo nội dung tự tạo, có tính điểm.
3	Quản lý nhóm học (class)	Tạo nhóm học, tham gia vào lớp để thi.
4	Thư viện đề (Library)	Danh sách đề luyện tập và nhóm học được lưu trữ và phân loại.
5	Tìm kiếm	Cho phép tìm bài học, lớp học, học phần, bài thi.
6	Hồ sơ cá nhân (Profile)	Hiển thị avatar, tên người dùng, cài đặt, lịch sử hoạt động.
7	Cài đặt ứng dụng (Setting)	Thay đổi mật khẩu người dùng, ảnh đại diện.
8	Tạo và quản lý lịch	Người dùng đánh dấu các ngày quan trọng (thi, kiểm tra).
9	Kết quả học tập	Thống kê điểm số, hiển thị bài đã học, bài thi và đánh giá.

3.2. Yêu cầu phi chức năng

Loại	Mô tả
Khả năng sử dụng (Usability)	Giao diện đơn giản, dễ dùng cho mọi đối tượng.
Hiệu năng (Performance)	Ứng dụng phản hồi nhanh, xử lý dữ liệu mượt trên thiết bị Android phổ thông.
Bảo mật (Security)	Dữ liệu cá nhân và thông tin người dùng được bảo mật và ẩn đi.
Mở rộng (Scalability)	Có thể tích hợp phân loại bài học, chia sẻ bộ câu hỏi trong tương lai.
Khả năng tương thích	Tương thích với thiết bị Android từ API 24 (Android 7.0) trở lên.
Dễ bảo trì	Sử dụng kiến trúc MVVM giúp bảo trì và nâng cấp module dễ dàng.

3.3. Sơ đồ Use Case (Tổng quát)

Use case: Đăng nhập	
Actor	Người dùng
Mục tiêu	Truy cập hệ thống
Tiền điều kiện	Đã đăng ký tài khoản
Luồng chính	- Nhập email và mật khẩu - Hệ thống kiểm tra thông tin Firebase - Đăng nhập thành công
Kết quả	Người dùng truy cập hệ thống

Use case: Tạo bài thi	
Actor	User
Mục tiêu	Tạo 1 bài thi mới cho class
Tiền điều kiện	Đã đăng nhập
Luồng chính	1. Vào chức năng “tạo bài thi” 2. Nhập tên bài thi, mô tả, danh sách câu hỏi 3. Nhấn lưu → dữ liệu lưu vào Firestore
Kết quả	Bài thi được lưu vào hệ thống

Use case: Làm bài kiểm tra	
Actor	User
Mục tiêu	Tham gia làm bài thi
Tiền điều kiện	Đã tham gia lớp / nhận được bài thi
Luồng chính	1. Chọn bài thi đã giao 2. Trả lời câu hỏi 3. Nhấn nộp bài 4. Kết quả được ghi lại trong Firestore
Kết quả	Hệ thống ghi nhận kết quả thi

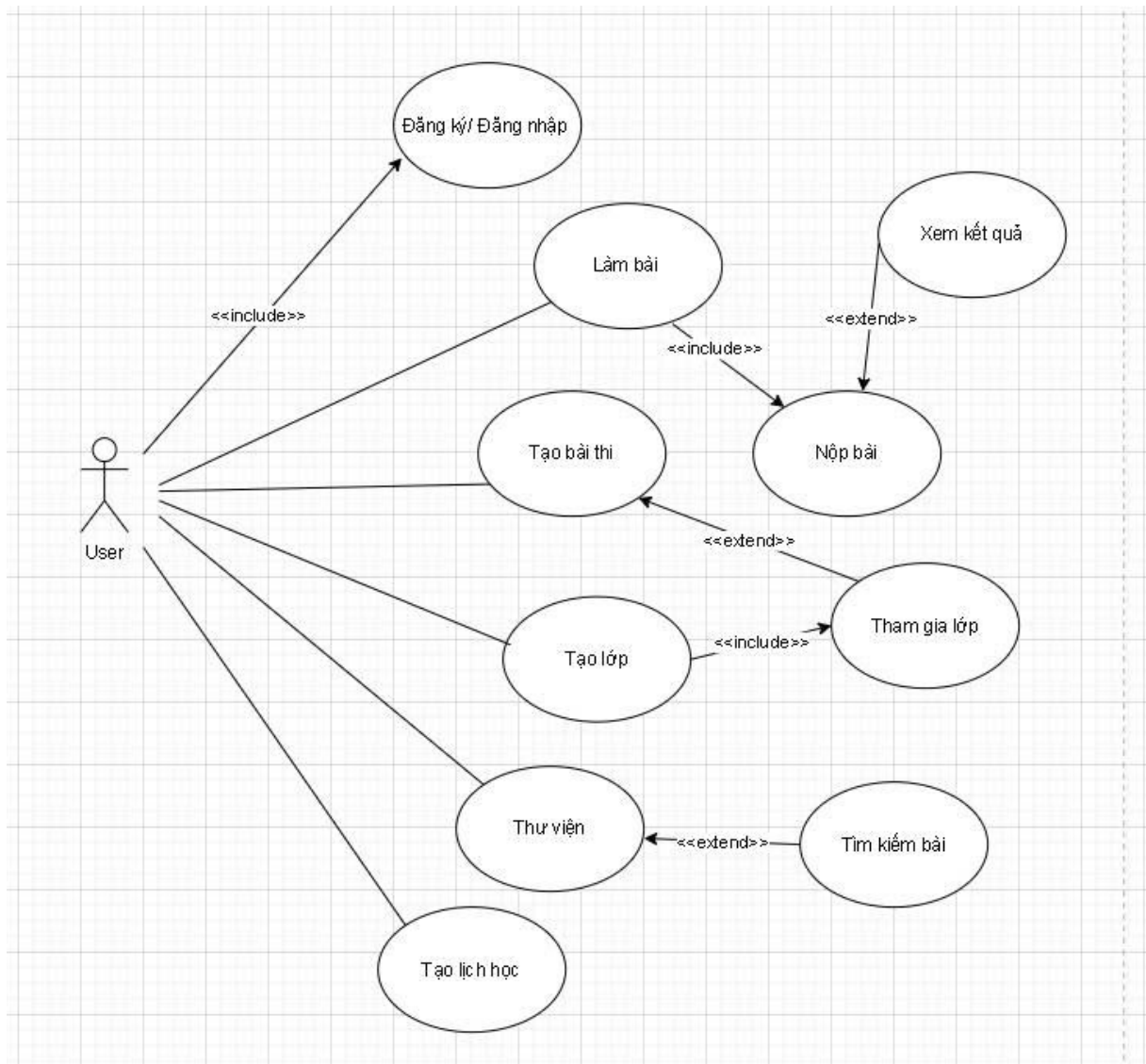
Use case: Tạo lớp học	
Actor	User
Mục tiêu	Tạo lớp mới
Luồng chính	1. Nhập tên lớp, mô tả 2. Nhấn tạo → sinh ra mã lớp → lưu vào Firestore
Kết quả	Tạo lớp hoàn tất

Use case: Tham gia lớp học	
Actor	User
Mục tiêu	Tham gia lớp học
Luồng chính	1. Nhập mã lớp 2. Kiểm tra mã 3. Lưu vào danh sách lớp trong Firestore
Kết quả	Tham gia thành công

Use case: Xem thư viện	
Actor	User
Mục tiêu	Truy cập tài liệu học tập
Luồng chính	1. Yêu cầu truy cập thư viện 2. Firestore trả về danh sách Quiz
Kết quả	Hiển thị các quiz

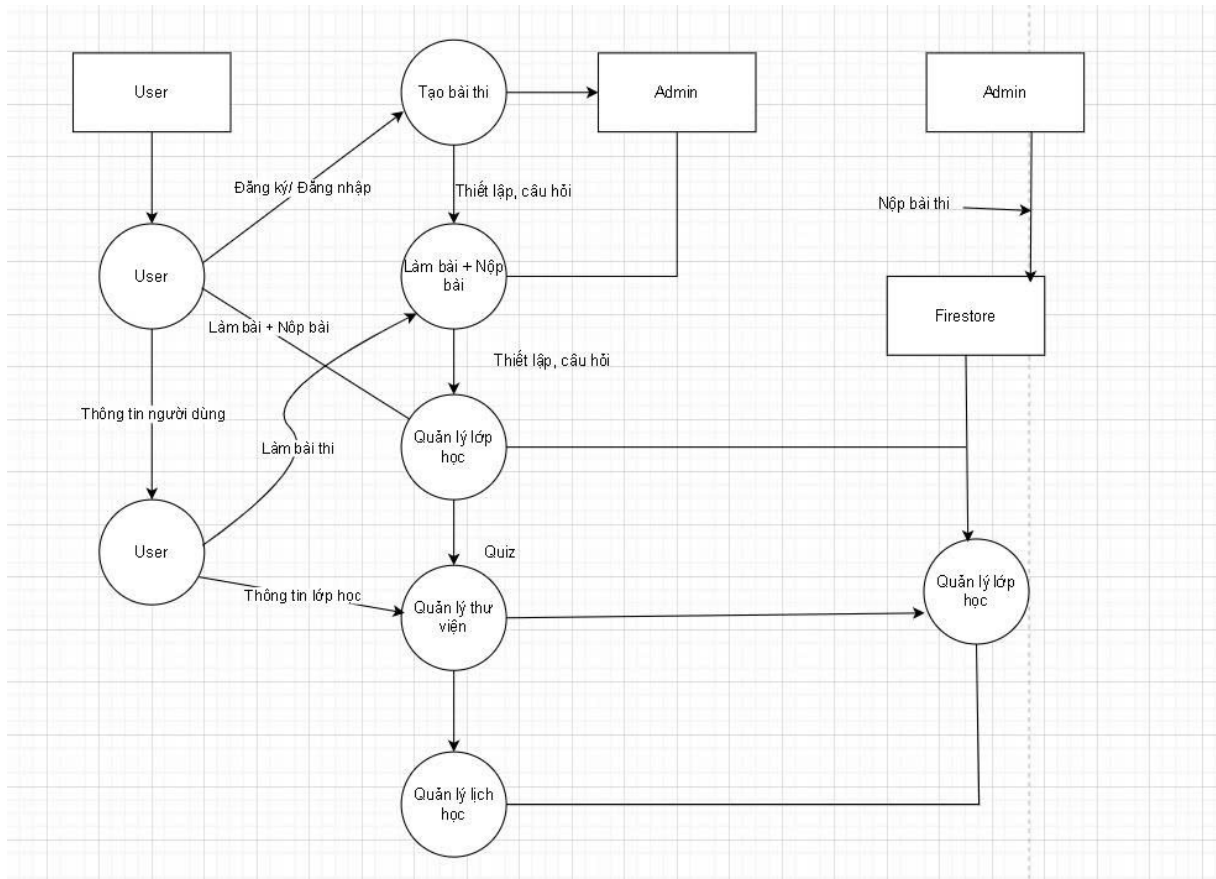
Use case: Quản lý Profile	
Actor	User
Mục tiêu	Cập nhật thông tin cá nhân
Luồng chính	1. Nhập tên, email, avatar 2. Lưu vào Firestore
Kết quả	Cập nhật thông tin thành công

Use case: Tạo lịch học	
Actor	User
Mục tiêu	Tạo lịch học cho bản thân
Luồng chính	1. Nhập tiêu đề 2. Nhập thời gian bắt đầu và kết thúc 3. Lưu vào Firestore
Kết quả	Tạo lịch học thành công



Hình 2: Sơ đồ tổng quát

3.4. Biểu đồ luồng dữ liệu (DFD - Level 0)



Hình 3: Biểu đồ luồng dữ liệu (DFD - Level 0)

CHƯƠNG 4. THIẾT KẾ HỆ THỐNG

4.1. Thiết kế UI/UX

Ứng dụng **Quiz Logic IQ** được thiết kế theo phong cách trẻ trung, hiện đại, sử dụng màu xanh làm tông màu chủ đạo nhằm tạo cảm giác thông minh, dễ chịu và khuyến khích sự tập trung trong quá trình học tập. Giao diện được phát triển theo hướng **mobile-first**, đảm bảo tối ưu trải nghiệm cho người dùng điện thoại Android.

Các màn hình chính bao gồm:

- **Màn hình chào (Splash & Start):** hiển thị logo ứng dụng, chào mừng người dùng.

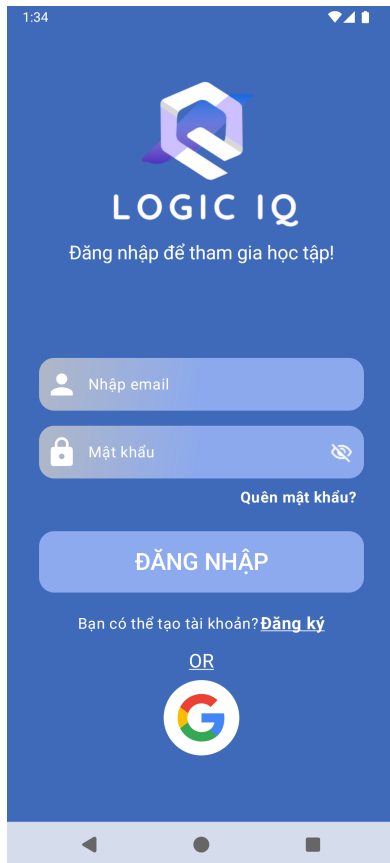


Hình 4: Màn hình chào

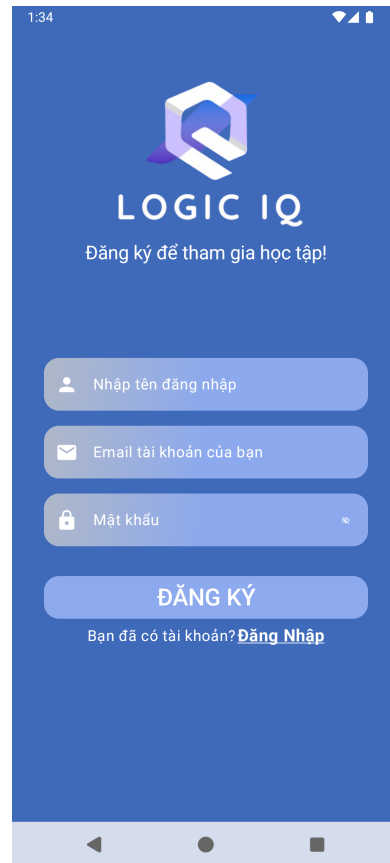


Hình 5: Màn hình Start

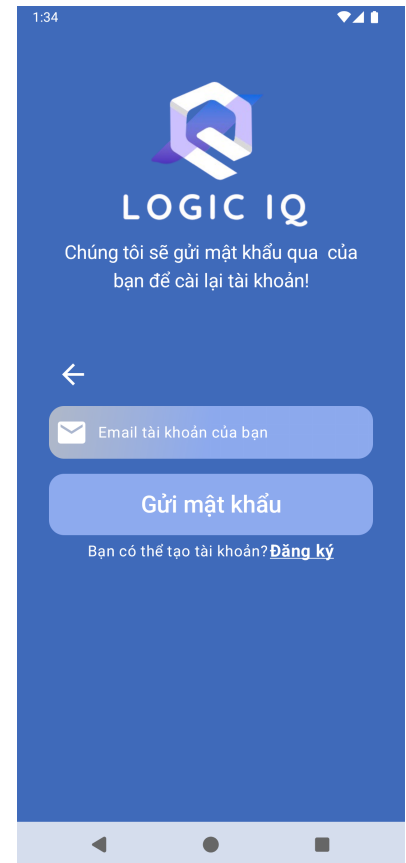
- **Màn hình Đăng nhập / Đăng ký / Reset password:** cho phép người dùng truy cập, tạo tài khoản hoặc khôi phục mật khẩu.



Hình 6: Màn hình Đăng nhập

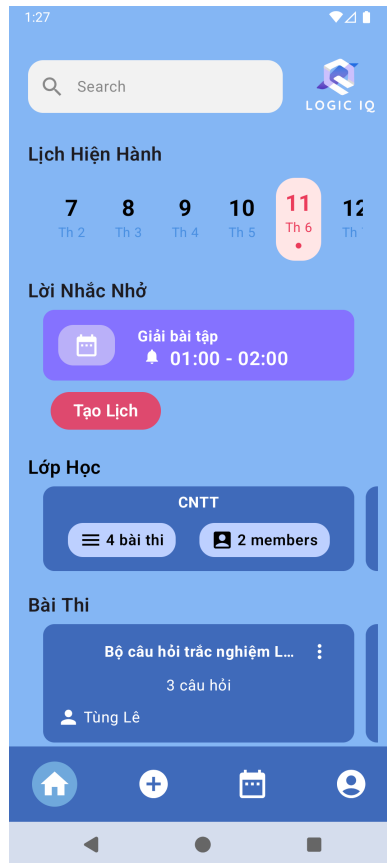


Hình 7: Màn hình Đăng ký



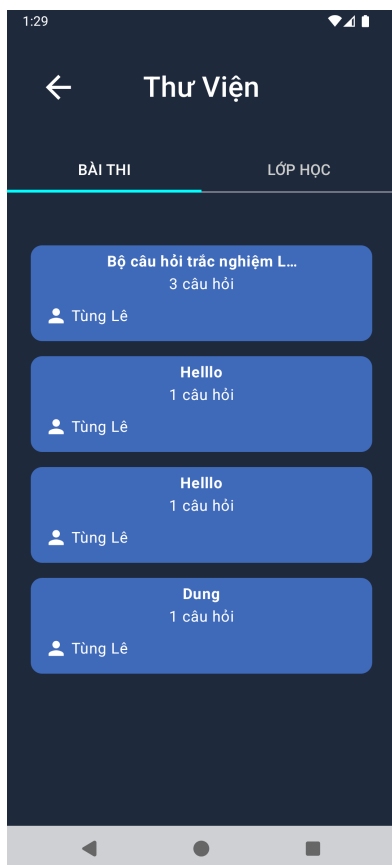
Hình 8: Màn hình Reset

- **Trang chủ (Home):** hiển thị các nội dung học tập như bài học, đề thi, lời nhắc, lớp học và công cụ tìm kiếm.

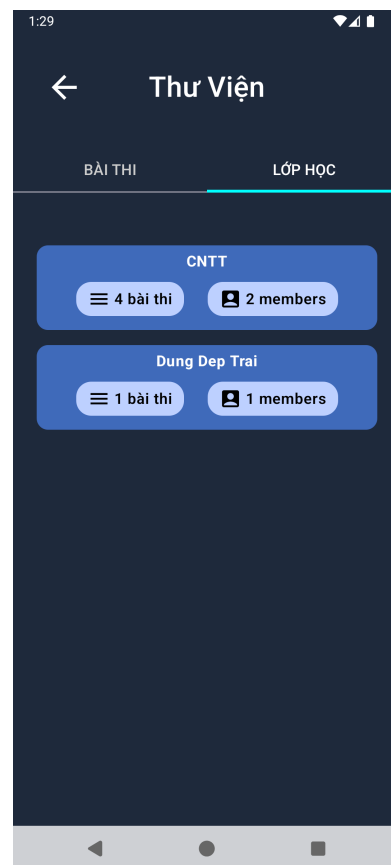


Hình 9: Giao diện màn hình Home

- **Thư viện đề:** nơi lưu trữ và phân loại các đề luyện tập và lớp học đã tham gia hoặc tạo.

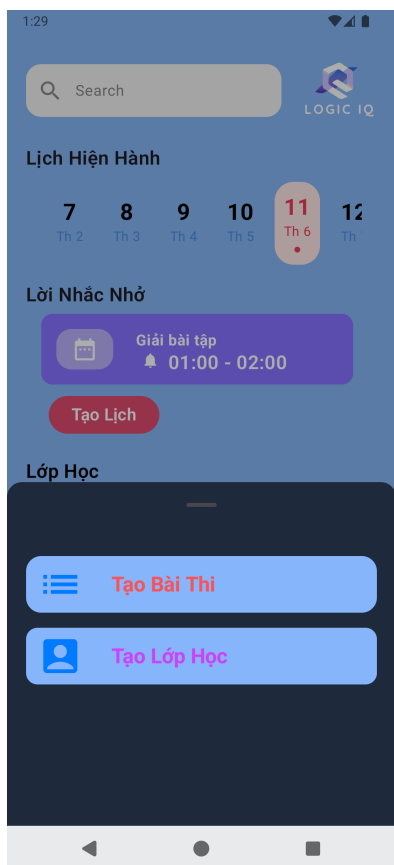


Hình 10: Màn hình Thư viện Exam

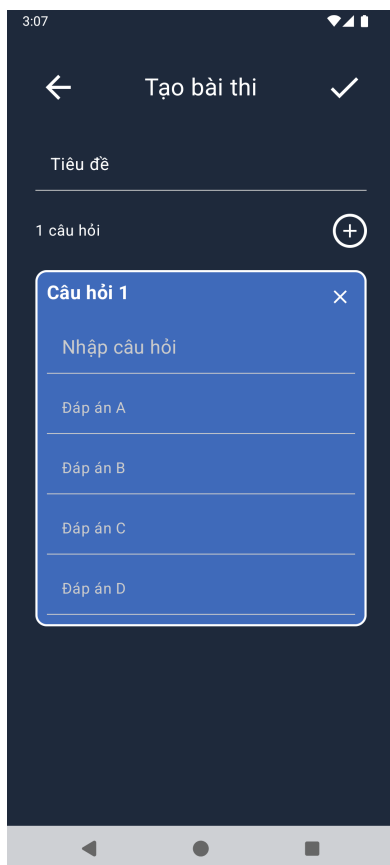


Hình 11: Màn hình Thư viện Class

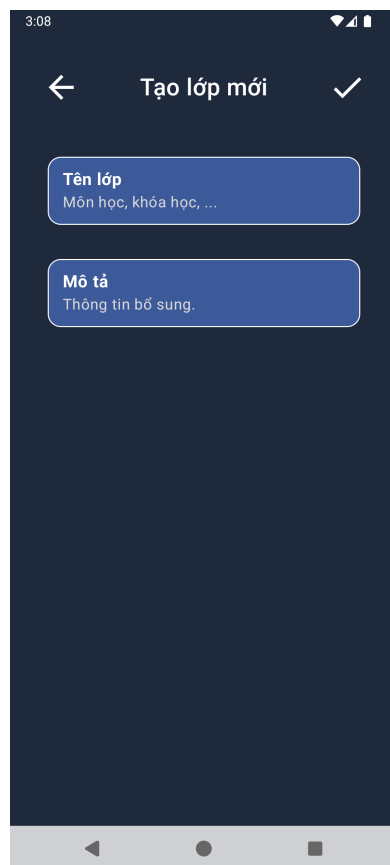
- **Tạo nội dung học tập:** cho phép người dùng tạo bài học, bài kiểm tra hoặc lớp học mới.



Hình 12: Màn hình Thêm mới

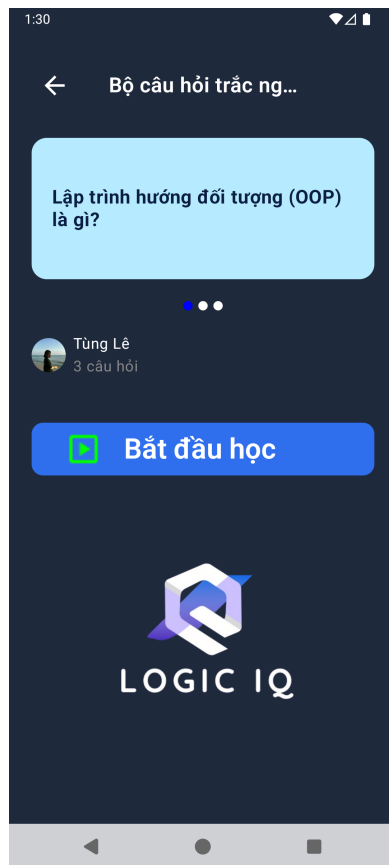


Hình 13: Màn hình Thêm bài thi



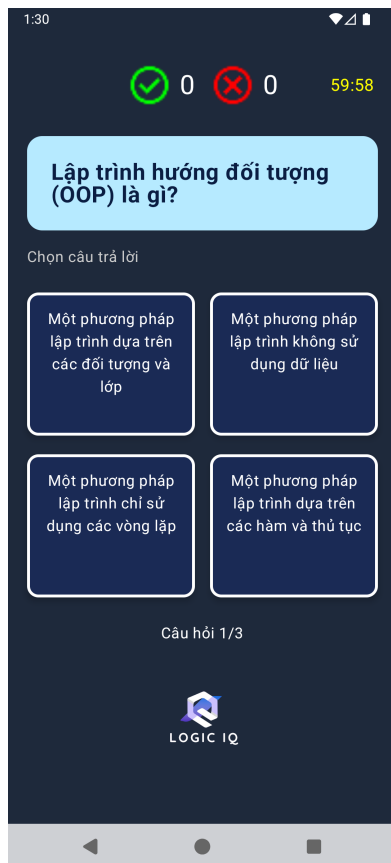
Hình 14: Màn hình Thêm lớp

- **Mô tả bài kiểm tra:** hiển thị thông tin chi tiết về bài thi đã chọn.

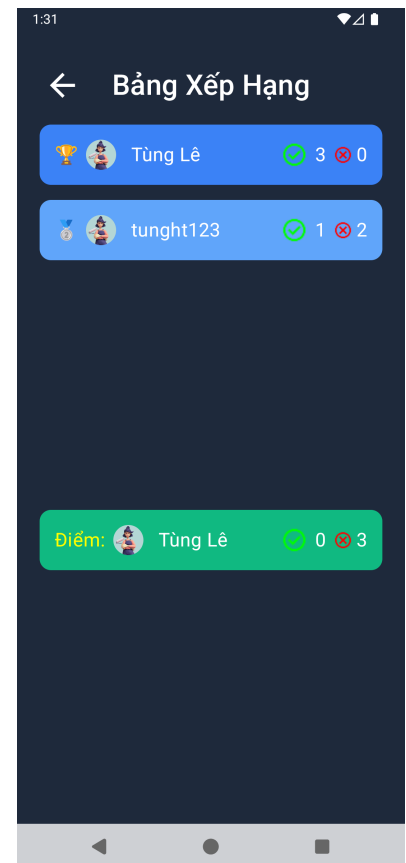


Hình 15: Giao diện màn hình chính bài kiểm tra

- **Màn hình kiểm tra và kết quả:** giao diện làm bài thi với tính thời gian và hiển thị kết quả sau khi hoàn thành.

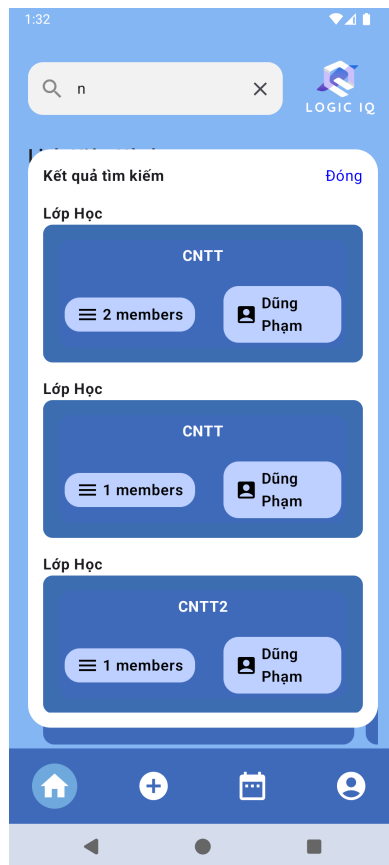


Hình 16: Màn hình làm bài thi



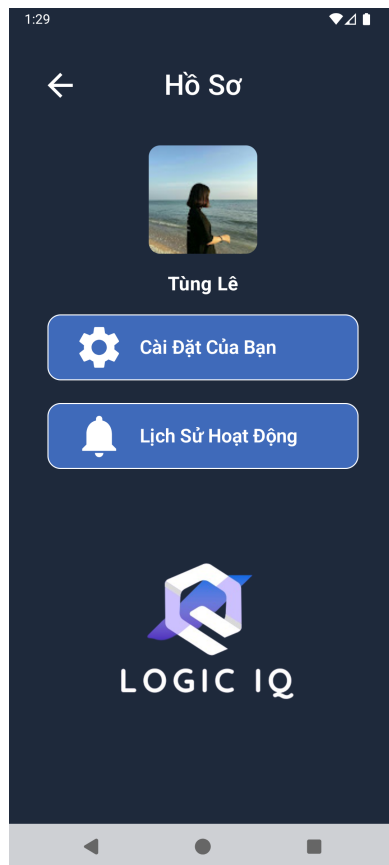
Hình 17: Màn hình bảng xếp hạng user khi làm bài

- **Tìm kiếm nội dung:** hỗ trợ tìm kiếm bài học, đề thi, lớp học theo từ khóa.



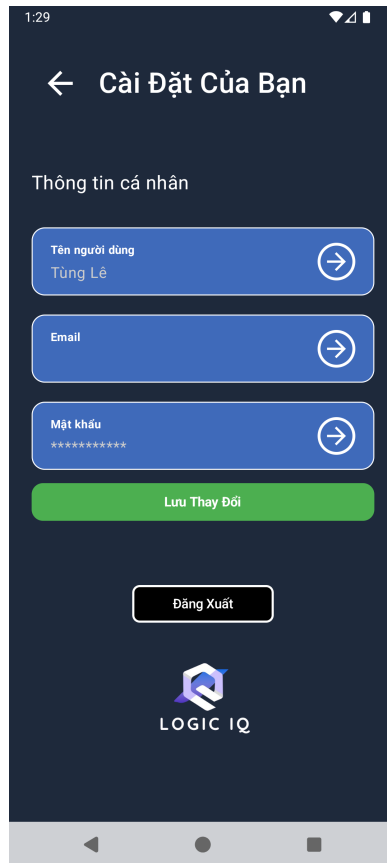
Hình 18: Giao diện màn hình tìm kiếm thông tin

- **Hồ sơ cá nhân (Profile):** hiển thị thông tin người dùng, lịch sử hoạt động, cài đặt tài khoản.



Hình 19: Giao diện màn hình hồ sơ của người dùng

- **Cài đặt hệ thống (Settings):** cho phép người dùng thay đổi mật khẩu, ảnh đại diện và các cài đặt cá nhân.



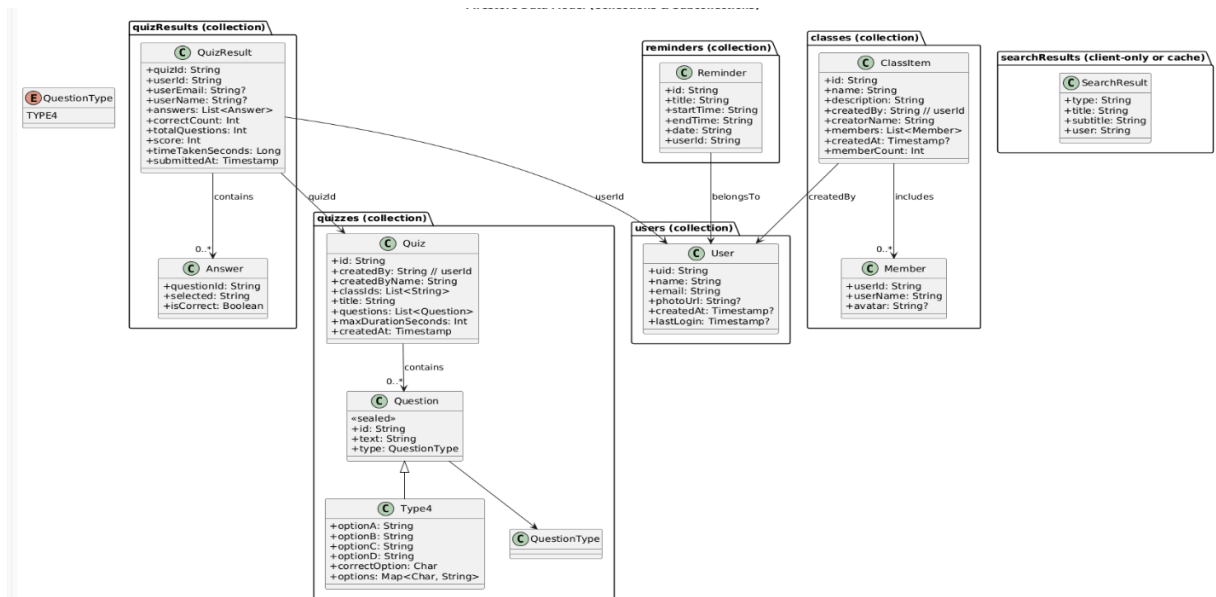
Hình 20: Giao diện màn hình cài đặt cá nhân

4.2. Thiết kế cơ sở dữ liệu (Firebase Realtime Database / Firestore)

Ứng dụng sử dụng **Firebase Firestore** để lưu trữ dữ liệu, với cấu trúc đơn giản và mở rộng linh hoạt:

Ghi chú:

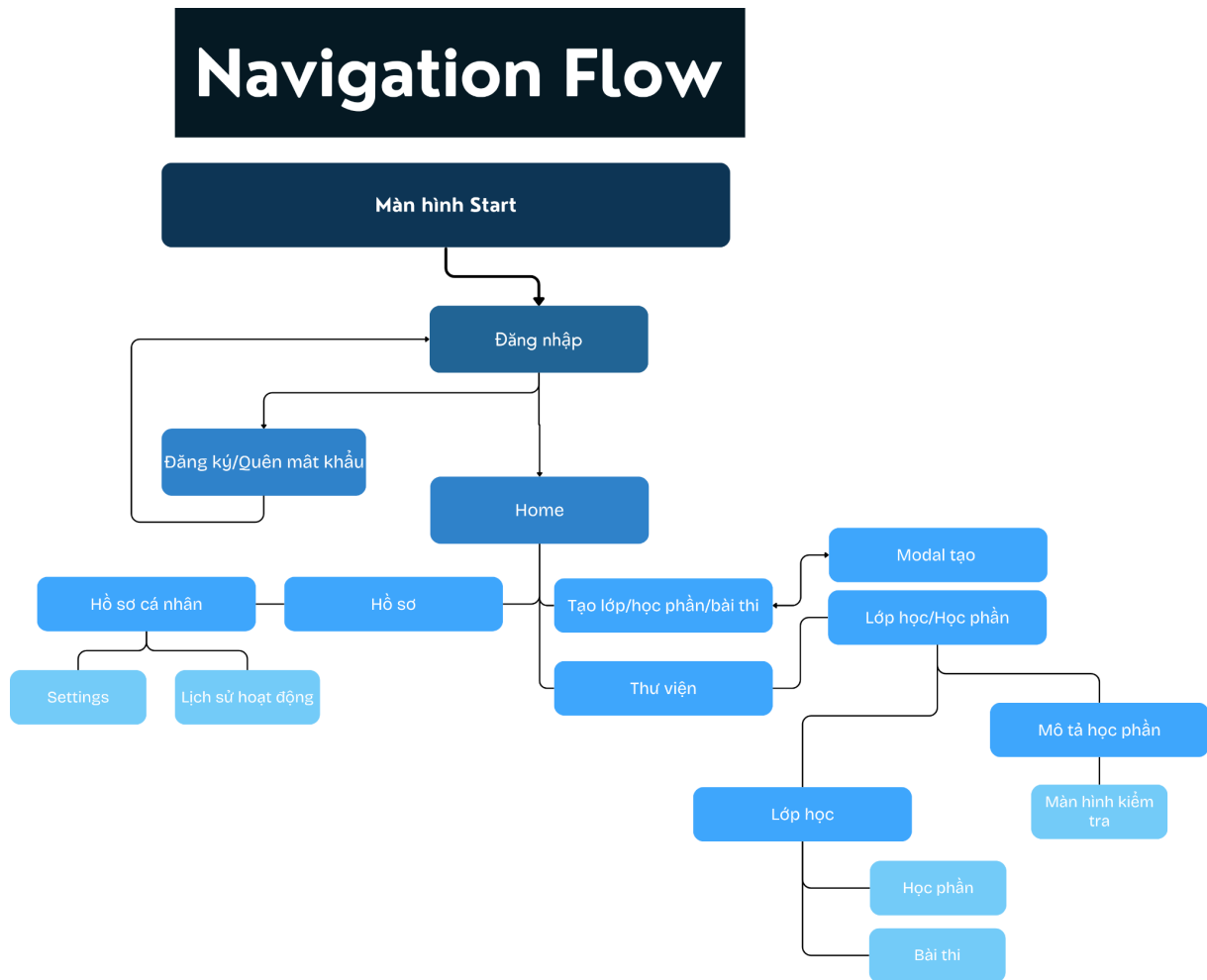
- Dữ liệu người dùng và lớp học liên kết bằng *uid*
- Câu hỏi được tách riêng để tái sử dụng



Hình 21: Thiết kế cơ sở dữ liệu trên Firebase.

4.3. Thiết kế luồng điều hướng (Navigation Flow)

Hệ thống sử dụng kiến trúc Navigation Compose để điều hướng giữa các màn hình.



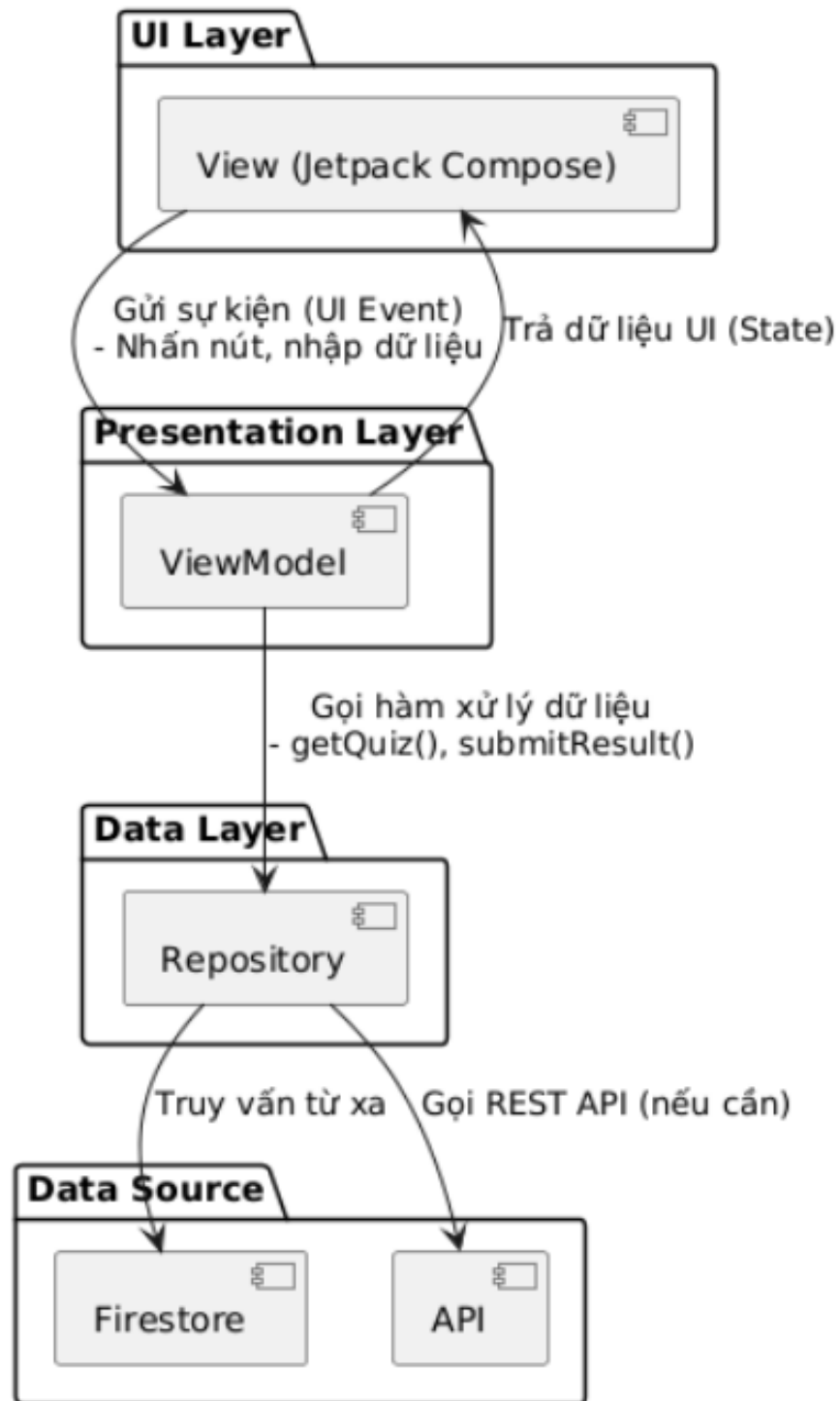
Hình 22: Thiết kế điều hướng (Navigation Flow).

4.4. Sơ đồ kiến trúc hệ thống

Áp dụng kiến trúc MVVM: View – ViewModel – Model

Trình bày sơ đồ tổng thể các tầng và cách chúng tương tác.

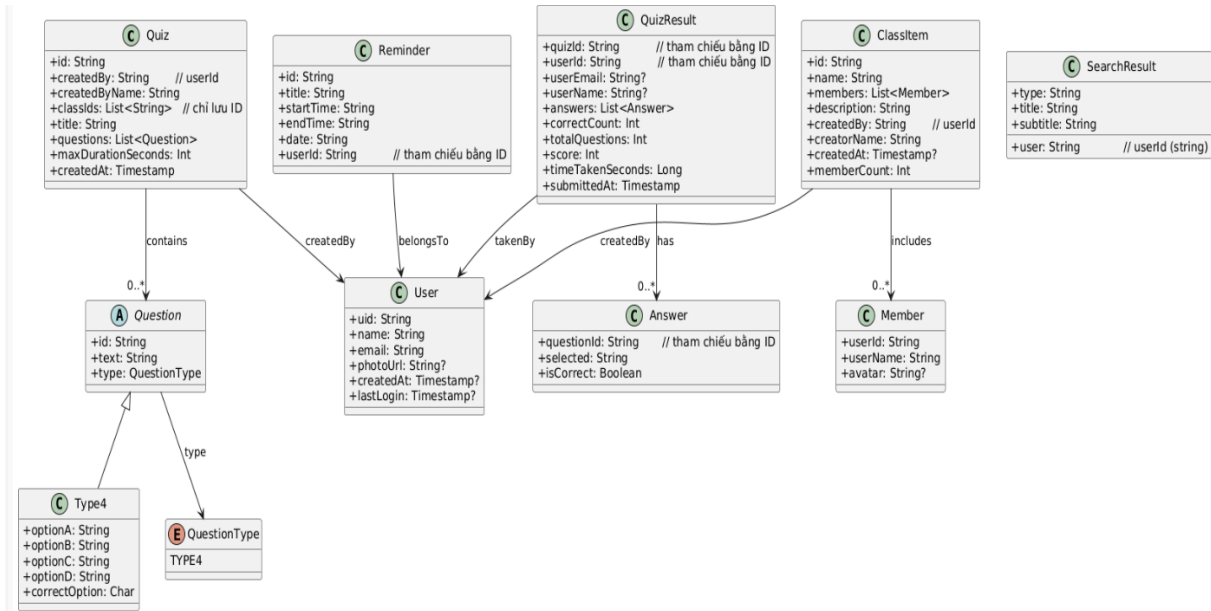
Kiến trúc hệ thống MVVM (Quiz Logic IQ App)



Hình 23: Sơ đồ kiến trúc hệ thống MVVM.

4.5. Sơ đồ lớp (Class Diagram)

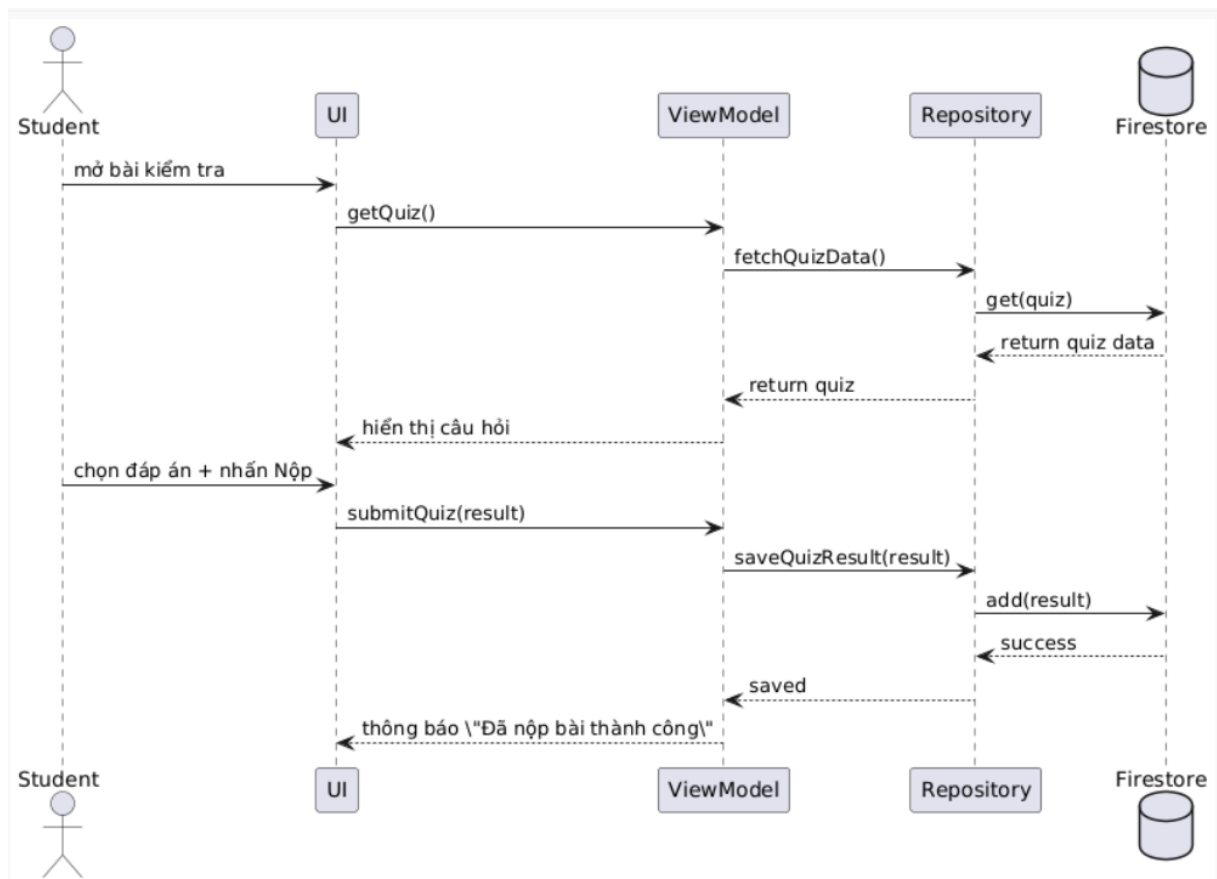
- UML biểu diễn các lớp chính: User, Quiz, Question, Answer, Class, QuizResult, Reminder, SearchResult
- Các thuộc tính và quan hệ giữa các lớp.



Hình 24: Sơ đồ lớp class diag.

4.6. Sơ đồ tuần tự (Sequence Diagram)

- Mô tả luồng hoạt động cho một chức năng tiêu biểu, như “Làm bài luyện tập”
- Tác nhân: User → UI → ViewModel → Repository → Database



Hình 25: Sơ đồ tuần tự xử lý luồng.

CHƯƠNG 5. TRIỂN KHAI VÀ CÀI ĐẶT

5.1. Môi trường phát triển

- **Android Studio:** Phiên bản khuyến nghị từ Android Studio 7 trở lên.
- **Ngôn ngữ lập trình:** Kotlin
- **Jetpack Compose version:** Tuỳ theo Android Studio, ví dụ: `compose-bom:2024.01.00`
- **Gradle version:** Tối thiểu 8.0 trở lên
- **minSdkVersion:** 21 (hoặc cao hơn tùy tính năng)
- **targetSdkVersion:** 34
- **Firebase SDK:** Tích hợp qua Firebase BoM

5.2. Thư viện sử dụng

```
// Core AndroidX
implementation(libs.androidx.core.ktx)
implementation(libs.androidx.lifecycle.runtime.ktx)
implementation(libs.androidx.activity.compose)

// Jetpack Compose
implementation(platform(libs.androidx.compose.bom))
implementation(libs.androidx.ui)
implementation(libs.androidx.ui.graphics)
implementation(libs.androidx.ui.tooling.preview)
implementation(libs.androidx.material3)

// Navigation
implementation(libs.androidx.navigation.compose)

// DateTime hỗ trợ
implementation(libs.threetenbp)

// Firebase
implementation(platform("com.google.firebase:firebase-bom:33.16.0"))
implementation("com.google.firebase:firebase-analytics")
implementation("com.google.firebase:firebase-auth:22.3.1")
implementation("com.google.firebase:firebase-firestore:24.11.0")
```



```
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-play-
services:1.8.0")
implementation("com.google.android.gms:play-services-auth:20.7.0")

// Test
testImplementation(libs.junit)
androidTestImplementation(libs.androidx.junit)
androidTestImplementation(libs.androidx.espresso.core)
androidTestImplementation(platform(libs.androidx.compose.bom))
androidTestImplementation(libs.androidx.ui.test.junit4)
debugImplementation(libs.androidx.ui.tooling)
debugImplementation(libs.androidx.ui.test.manifest)
```

5.3. Các bước triển khai ứng dụng

1. Cài đặt Android Studio và SDK:

- Mở Android Studio → Tạo project mới → Chọn *Empty Compose Activity*.

2. Thêm Firebase vào project:

- Truy cập Firebase Console → Tạo Project mới.
- Thêm Android app → Nhập tên package → Tải `google-services.json`.
- Thêm file này vào thư mục `app/`.

3. Cấu hình Gradle:

- Trong `settings.gradle` thêm:

```
pluginManagement {
    repositories {
        gradlePluginPortal()
        google()
        mavenCentral()
    }
}
```

- Trong `build.gradle (app)` thêm:

```
plugins {
    id 'com.google.gms.google-services'
}
```

4. Đồng bộ Gradle:

- Nhấn “Sync Now” để tải tất cả các dependencies.

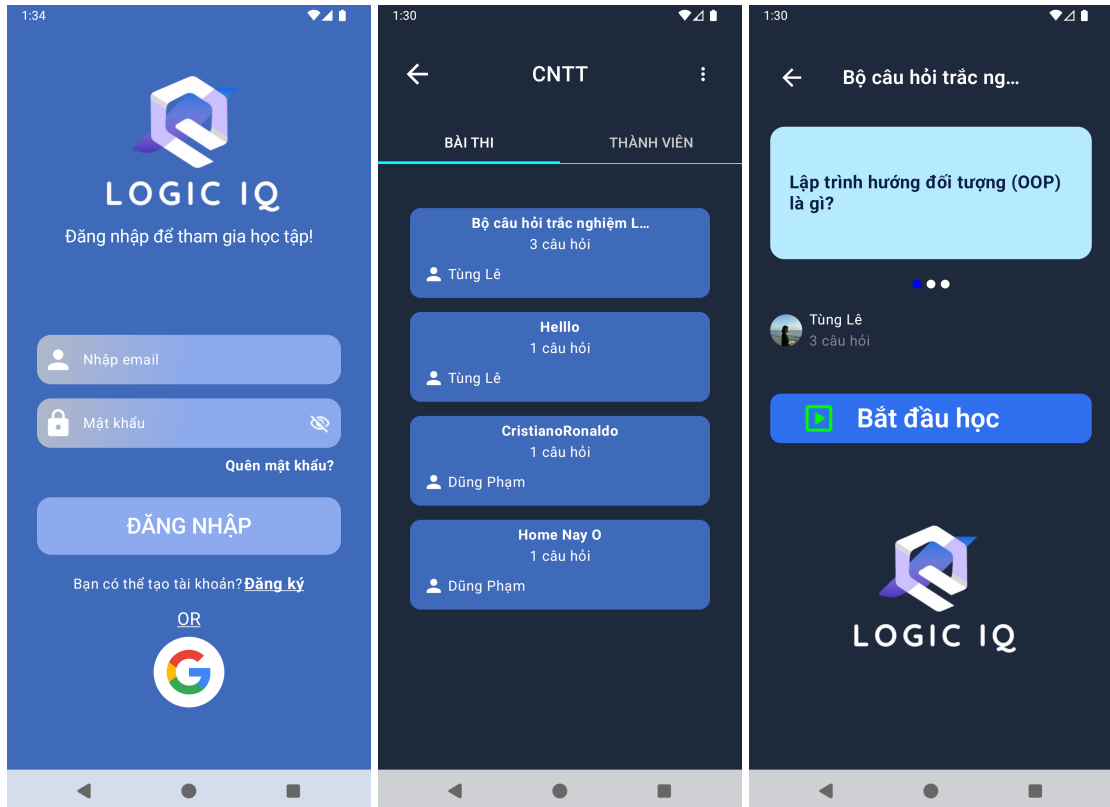
5. Build và chạy ứng dụng:

- Chọn thiết bị ảo hoặc thiết bị thật.
- Nhấn Run hoặc dùng phím tắt `Shift + F10`.

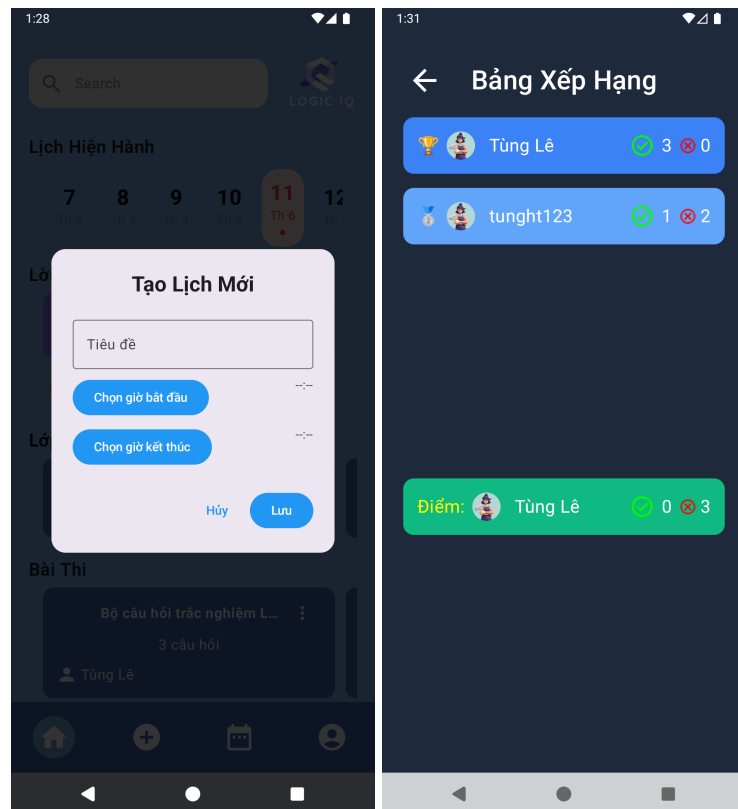
CHƯƠNG 6. KẾT QUẢ THỰC NGHIỆM

6.1. Hình ảnh giao diện thực tế

Dưới đây là một số ảnh minh họa về giao diện của ứng dụng Logic IQ sau khi triển khai:



Hình 26: Màn hình đăng nhập, danh sách bài tập, giao diện làm bài.



Hình 27: Thống kê kết quả và nhắc lịch luyện tập.

(Hình ảnh được chụp từ ứng dụng chạy thực tế trên thiết bị Android hoặc Emulator.)

6.2. Các chức năng đã thực hiện được

- Đăng ký/đăng nhập người dùng (Firebase Authentication)
- Lưu và tải dữ liệu người dùng từ Firestore
- Danh sách câu hỏi và bài tập logic theo cấp độ
- Giao diện luyện tập trực quan, hiển thị kết quả đúng/sai
- Thống kê điểm số, lịch sử luyện tập
- Nhắc lịch luyện tập bằng thông báo đẩy (AlarmManager + Notification)
- Điều hướng mượt mà giữa các màn hình bằng Navigation Compose
- Giao diện hoàn toàn sử dụng Jetpack Compose, không dùng XML

6.3. Demo sản phẩm

Nếu buổi bảo vệ diễn ra offline, nhóm sẽ chuẩn bị:

- Thiết bị Android có sẵn ứng dụng để demo trực tiếp
- Trình diễn: đăng nhập → luyện tập → thống kê → nhận thông báo nhắc luyện tập

CHƯƠNG 7. ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

7.1. Ưu điểm của ứng dụng

- Giao diện hiện đại, thân thiện với người dùng nhờ Jetpack Compose
- Ứng dụng hoạt động mượt mà, phản hồi nhanh
- Có khả năng mở rộng thêm cấp độ, câu hỏi, chủ đề logic
- Sử dụng Firebase nên dễ quản lý người dùng và dữ liệu
- Hỗ trợ cả học tập và giải trí có tính giáo dục

7.2. Hạn chế

- Bộ câu hỏi còn giới hạn, chưa có chế độ ngẫu nhiên hoặc tự tạo câu hỏi
- Giao diện chưa hỗ trợ chế độ tối (Dark Mode)
- Chưa có chức năng thi đấu.

7.3. Hướng phát triển trong tương lai

- Phát triển hệ thống thử thách mỗi ngày và bảng xếp hạng
- Cho phép người dùng tự tạo câu hỏi và chia sẻ cho người khác
- Thêm nhiều chủ đề: toán học, suy luận hình học, ngôn ngữ logic,...
- Đồng bộ hóa tiến trình học trên nhiều thiết bị
- Thêm Dark Mode và hỗ trợ nhiều ngôn ngữ (đa ngữ)

CHƯƠNG 8. KẾT LUẬN

Dự án Logic IQ đã được xây dựng thành công với các chức năng cốt lõi nhằm hỗ trợ người dùng luyện tập tư duy logic một cách trực quan và hiệu quả. Ứng dụng không chỉ phục vụ mục tiêu học tập mà còn là công cụ giải trí bổ ích.

Thông qua việc áp dụng các công nghệ hiện đại như Kotlin, Jetpack Compose, Firebase, nhóm đã học hỏi được nhiều kỹ năng quan trọng trong phát triển ứng dụng Android, đồng thời cải thiện khả năng làm việc nhóm, lập trình bất đồng bộ và xử lý dữ liệu thời gian thực.

CHƯƠNG 9. TÀI LIỆU THAM KHẢO

1. Android Developers - Jetpack Compose: <https://developer.android.com/jetpack/compose>
2. Kotlin Language Documentation: <https://kotlinlang.org/docs/home.html>
3. Firebase Documentation: <https://firebase.google.com/docs>
4. Google Codelabs - Android Compose Basics: <https://developer.android.com/codelabs>
5. Hilt for Dependency Injection: <https://developer.android.com/training/dependency-injection/hilt-android>